



InfoControl

Sistema de Gestão Comercial

Loja de Informática · Aplicação Web com API REST

DOCUMENTAÇÃO TÉCNICA · ENTREGA 3 — SISTEMA COMPLETO

Iorran Valença · Gabriel Pereira

1. Introdução

Com o crescimento do setor de tecnologia e a constante demanda por equipamentos e serviços de informática, torna-se essencial que empresas desse segmento possuam ferramentas eficientes para gerenciar suas operações comerciais. A organização adequada de dados como clientes, produtos, vendas e estoque impacta diretamente na qualidade do atendimento e na tomada de decisões.

Neste contexto, o projeto propõe o desenvolvimento de um sistema web completo para automatizar e integrar os principais processos de uma loja de informática, com API REST, autenticação por token, interface web e relatórios gerenciais.

2. Descrição do Sistema

O **InfoControl** é uma aplicação web voltada para a gestão comercial de uma loja de informática. Permite o controle de clientes, produtos, estoque e vendas, além de oferecer um painel de relatórios gerenciais que auxilia na análise do desempenho do negócio.

A aplicação é dividida em uma **API REST** (que concentra as regras de negócio e a persistência) e uma **interface web** que consome essa API diretamente do navegador. Conta com autenticação por token **JWT**, recuperação de senha por e-mail, controle de acesso por perfil de usuário e armazenamento em banco de dados relacional.

3. Contexto do Negócio

A loja de informática comercializa produtos como computadores, notebooks, periféricos, componentes, monitores e acessórios tecnológicos para clientes finais. Muitas vezes o controle dessas operações é feito de forma manual ou com ferramentas limitadas, o que gera inconsistências, perda de informações e dificuldades no controle de estoque.

O sistema centraliza todas essas operações em uma única plataforma, proporcionando maior organização, segurança e eficiência — com recursos próprios do varejo de tecnologia, como classificação de produtos por categoria, controle de marca e garantia, e relatórios de itens mais vendidos e de reposição de estoque.

4. Objetivo do Sistema

Desenvolver um sistema web que permita gerenciar as operações comerciais de uma loja de informática, incluindo cadastro de clientes, catálogo de produtos com controle de estoque, registro de vendas, autenticação e recuperação de usuários, controle de acesso por perfil e geração de relatórios gerenciais — tudo integrado por meio de uma API REST consumida por uma interface web.

5. Identidade Visual

O sistema possui nome e identidade próprios. O logotipo **InfoControl** combina um círculo laranja com a marca nominativa, e a interface adota uma paleta quente e minimalista, com tipografia serifada nos títulos e cartões claros sobre um fundo creme.

ELEMENTO	DEFINIÇÃO
----------	-----------

Nome	InfoControl — Sistema de Gestão Comercial
Cor principal	Laranja #FF751F (logo, botões, gráficos, destaques)
Fundo	Off-white / creme #F0EEE6
Texto	Quase-preto quente #2B2A27
Estilo	Cartões claros, bordas suaves, títulos serifados — visual limpo e acolhedor

6. Requisitos Funcionais

CÓDIGO	DESCRIÇÃO
RF01	Permitir o login de usuários com autenticação por token JWT.
RF02	Permitir a recuperação de senha por e-mail (token temporário). novo
RF03	Permitir o cadastro, edição, listagem e exclusão de clientes (respeitando regras de negócio).
RF04	Permitir o cadastro, edição, listagem e exclusão de produtos com atributos de catálogo: marca , categoria e garantia . atualizado
RF05	Permitir a busca de produtos por nome/marca e o filtro por categoria. novo
RF06	Permitir o registro de vendas com múltiplos itens, com carrinho preservado ao navegar entre telas.
RF07	Calcular automaticamente o valor total da venda.
RF08	Atualizar o estoque automaticamente após a realização da venda.
RF09	Impedir a venda de produtos com estoque insuficiente ou sem itens.
RF10	Listar produtos com estoque no/abaixo do mínimo (reposição). novo
RF11	Gerar relatório de vendas por período, com indicadores (total, nº de vendas, ticket médio).
RF12	Permitir a consulta de vendas por cliente.
RF13	Gerar gráfico anual de vendas (representação gráfica mês a mês).
RF14	Gerar relatório de produtos mais vendidos (best-sellers). novo
RF15	Exibir os detalhes de cliente, produto e venda em cartão com animação. novo
RF16	Controlar o acesso por perfil (ADMIN e FUNCIONARIO); exclusões restritas ao perfil ADMIN.
RF17	Disponibilizar interface web que consome a API REST.

7. Requisitos Não Funcionais

CÓDIGO	DESCRIÇÃO
RNF01	Arquitetura em camadas, organizada nos módulos config, api e web.
RNF02	Interface web em Django, consumindo a API via JavaScript (fetch).
RNF03	API REST com retorno em JSON em todas as solicitações.
RNF04	Autenticação baseada em token JWT, com tempo de expiração.
RNF05	Senhas armazenadas de forma criptografada (hash PBKDF2, padrão do Django).
RNF06	Banco de dados relacional (SQLite, portátil para PostgreSQL).
RNF07	Tratamento global de exceções, com resposta de erro padronizada.
RNF08	Identidade visual própria (logotipo e esquema de cores).

RNF09	Versionamento no GitHub, com histórico de commits organizado.
RNF10	Documentação técnica detalhada e testes automatizados (18 casos).
RNF11	Código limpo, comentado e de fácil manutenção e escalabilidade.

8. Arquitetura

O sistema adota uma **arquitetura em camadas**, separando responsabilidades para facilitar a manutenção e a evolução. A interface é totalmente **desacoplada** do backend: o front-end consome a API REST via fetch, o que comprova o funcionamento independente da API (que poderia servir também um aplicativo móvel).

CAMADA	ONDE	RESPONSABILIDADE
Apresentação	web/ (templates + JS)	Telas HTML que consomem a API.
API / Controle	api/views.py, api/urls.py	Recebe as requisições HTTP e devolve JSON.
Negócio (Serviço)	api/services.py	Regras de venda, estoque e relatórios.
Persistência	api/models.py (ORM)	Mapeamento objeto-relacional com o banco.

Estrutura de pastas

```
infocontrol/
├─ manage.py · requirements.txt
├─ config/          # Configuração do projeto (settings, urls, wsgi)
├─ api/             # API REST (regras de negócio)
│   ├── models.py      # Cliente, Produto, Venda, ItemVenda, PerfilUsuario
│   ├── serializers.py  # Conversão e validação JSON ⇄ modelo
│   ├── services.py    # Regras de negócio (vendas e relatórios)
│   ├── views.py       # Endpoints REST
│   ├── auth.py        # Login JWT, perfis e recuperação de senha
│   ├── exceptions.py  # Exceções de negócio + handler global
│   └── tests.py       # 18 testes automatizados
├─ web/             # Interface web (consome a API via JavaScript)
│   └─ templates/web/  # login, clientes, produtos, vendas, relatórios
```

9. Padrões de Projeto Utilizados

PADRÃO	APLICAÇÃO NO INFOCONTROL
Arquitetura em Camadas	Separação entre apresentação, API, negócio e persistência.
Service Layer	Regras de negócio centralizadas em services.py, fora das views.
Serializer / DTO	serializers.py valida e converte os dados de entrada e saída.
Repository (via ORM)	O ORM do Django abstrai o acesso ao banco de dados.
Exception Handler Global	Todos os erros saem no formato padrão {"erro": ...}.
MVT (Model-View-Template)	Padrão do Django na entrega das páginas web.
RBAC (Permission Classes)	Controle de acesso por perfil (exclusões restritas ao ADMIN).

10. Modelo de Domínio

O sistema é composto pelas seguintes entidades principais e seus relacionamentos.

Usuario / PerfilUsuario

Responsável pela autenticação e operação do sistema. Possui um **perfil** (ADMIN ou FUNCIONARIO).

Cliente

Consumidor da loja. Identificado por CPF único.

Produto

Item do catálogo, com marca, categoria, preço, estoque e garantia.

Venda

Transação comercial registrada por um usuário para um cliente.

ItemVenda

Produto incluído em uma venda, com quantidade e subtotal.

Relacionamentos

- Um **usuário** registra várias vendas.
- Um **cliente** pode realizar várias vendas.
- Uma **venda** possui vários itens de venda.
- Cada **item de venda** está associado a um produto.

11. Diagrama de Classes

Estrutura do sistema sob a perspectiva orientada a objetos, com atributos e principais operações.



12. Modelo Lógico do Banco de Dados

O banco é composto por cinco tabelas, com chaves primárias (**PK**), estrangeiras (**FK**) e restrições de unicidade (**UK**). A tabela produtos foi ampliada com os campos **marca**, **categoria** e **garantia_meses**.



13. Regras de Negócio

Clientes

- CPF não pode ser duplicado.
- E-mail deve ser válido.
- Cliente não pode ser removido se possuir vendas registradas.

Produtos

- Preço, estoque e garantia não podem ser negativos.
- Cada produto pertence a uma categoria controlada (Notebooks, Computadores, Periféricos, Componentes, Monitores, Armazenamento, Redes, Acessórios, Outros).
- O sistema controla o estoque mínimo e sinaliza itens para reposição.
- Produto vinculado a vendas não pode ser removido.

Vendas

- Não é permitido registrar venda sem itens ou com estoque insuficiente.
- O valor total é calculado automaticamente.
- O estoque é atualizado após a venda.

Usuários e Segurança

- Senha armazenada com hash; autenticação via token JWT com expiração.
- Recuperação de senha por e-mail, usando token temporário e seguro.
- Controle de acesso por perfil — exclusões restritas ao perfil ADMIN.

14. API REST — Endpoints

Todas as respostas são em JSON. Salvo login e recuperação de senha, as rotas exigem o cabeçalho Authorization: Bearer <token>.

MÉTODO / ENDPOINT	DESCRIÇÃO
POST /api/auth/login/	Login → retorna access, refresh e perfil
POST /api/auth/refresh/	Renova o token de acesso
GET /api/auth/me/	Dados do usuário logado
POST /api/auth/recuperar-senha/	Envia e-mail de recuperação novo
POST /api/auth/redefinir-senha/	Redefine a senha (uid + token) novo
GET · POST /api/clientes/	Lista / cadastra clientes
GET · PUT · DELETE /api/clientes/{id}/	Detalha / edita / remove cliente
GET · POST /api/produtos/	Lista / cadastra produtos
GET · PUT · DELETE /api/produtos/{id}/	Detalha / edita / remove produto
GET /api/produtos/estoque-baixo/	Produtos no/abaixo do estoque mínimo novo
GET · POST /api/vendas/	Lista / registra vendas
GET /api/vendas/{id}/	Detalha uma venda
GET /api/relatorios/vendas/?inicio=&fim=	Vendas por período
GET /api/relatorios/vendas-cliente/{id}/	Vendas de um cliente
GET /api/relatorios/vendas-anuais/?ano=	Total mês a mês (gráfico)
GET /api/relatorios/mais-vendidos/	Produtos mais vendidos novo

15. Segurança

- **Autenticação JWT** (access + refresh) com tempo de expiração configurável.
- **Senhas com hash** (PBKDF2, padrão do Django) — nunca armazenadas em texto.
- **Recuperação de senha por e-mail** com token temporário; resposta genérica que não revela se o e-mail existe.
- **Controle de acesso por perfil** (ADMIN / FUNCIONARIO); apenas ADMIN pode excluir registros.
- Todas as rotas da API são protegidas por padrão.

16. Script de Criação do Banco

Script alinhado ao modelo lógico atual, incluindo os novos campos de catálogo da tabela produtos e as restrições de integridade.

```
CREATE TABLE usuarios (  
    id BIGINT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,  
    username VARCHAR(50) NOT NULL UNIQUE,  
    password_hash VARCHAR(255) NOT NULL,  
    perfil VARCHAR(20) NOT NULL,  
    CONSTRAINT chk_usuario_perfil CHECK (perfil IN ('ADMIN', 'FUNCIONARIO'))  
);  
  
CREATE TABLE clientes (  
    id BIGINT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,  
    nome VARCHAR(100) NOT NULL,  
    cpf VARCHAR(14) NOT NULL UNIQUE,  
    email VARCHAR(100) NOT NULL,  
    telefone VARCHAR(20),  
    endereco VARCHAR(150),  
    CONSTRAINT chk_cliente_email CHECK (POSITION('@' IN email) > 1)  
);  
  
CREATE TABLE produtos (  
    id BIGINT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,  
    nome VARCHAR(100) NOT NULL,  
    descricao VARCHAR(255),  
    marca VARCHAR(50),  
    categoria VARCHAR(20) NOT NULL DEFAULT 'OUTROS',  
    preco NUMERIC(10,2) NOT NULL,  
    quantidade_estoque INT NOT NULL DEFAULT 0,  
    estoque_minimo INT NOT NULL DEFAULT 0,  
    garantia_meses INT NOT NULL DEFAULT 12,  
    CONSTRAINT chk_produto_preco CHECK (preco >= 0),  
    CONSTRAINT chk_produto_quantidade CHECK (quantidade_estoque >= 0),  
    CONSTRAINT chk_produto_estoque_minimo CHECK (estoque_minimo >= 0),  
    CONSTRAINT chk_produto_garantia CHECK (garantia_meses >= 0),  
    CONSTRAINT chk_produto_categoria CHECK (categoria IN  
        ('NOTEBOOK', 'COMPUTADOR', 'PERIFERICO', 'COMPONENTE', 'MONITOR',  
        'ARMAZENAMENTO', 'REDE', 'ACESSORIO', 'OUTROS'))  
);  
  
CREATE TABLE vendas (  
    id BIGINT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,  
    data_venda TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    cliente_id BIGINT NOT NULL,  
    usuario_id BIGINT NOT NULL,  
    valor_total NUMERIC(10,2) NOT NULL DEFAULT 0,  
    CONSTRAINT fk_vendas_cliente FOREIGN KEY (cliente_id) REFERENCES clientes(id),  
    CONSTRAINT fk_vendas_usuario FOREIGN KEY (usuario_id) REFERENCES usuarios(id),  
    CONSTRAINT chk_venda_valor_total CHECK (valor_total >= 0)  
);  
  
CREATE TABLE itens_venda (  
    id BIGINT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,  
    venda_id BIGINT NOT NULL,  
    produto_id BIGINT NOT NULL,
```

```
quantidade INT NOT NULL,  
preco_unitario NUMERIC(10,2) NOT NULL,  
subtotal NUMERIC(10,2) NOT NULL,  
CONSTRAINT fk_itens_venda_venda FOREIGN KEY (venda_id) REFERENCES vendas(id) ON DELETE CASCADE,  
CONSTRAINT fk_itens_venda_produto FOREIGN KEY (produto_id) REFERENCES produtos(id),  
CONSTRAINT chk_item_quantidade CHECK (quantidade > 0),  
CONSTRAINT chk_item_preco_unitario CHECK (preco_unitario >= 0),  
CONSTRAINT chk_item_subtotal CHECK (subtotal >= 0)  
);
```

17. Tecnologias e Execução

Tecnologias: Python 3.12 · Django 6 · Django REST Framework · SimpleJWT (JWT) · SQLite · Chart.js (gráficos) · HTML/CSS/JavaScript.

Como executar (primeira vez)

```
python -m venv venv
venv\Scripts\activate          # Windows
pip install -r requirements.txt
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
# Acesse http://127.0.0.1:8000/
```

Testes automatizados: `python manage.py test` (18 casos cobrindo autenticação, CRUD, catálogo, regras de venda, relatórios e recuperação de senha).

Repositório GitHub: _____ (inserir o link do repositório)

18. Equipe

Iorran Valença · Gabriel Pereira